

3.9 FetchType and Orphan Removal in Relational Queries, N+1 Query Optimization

• FetchType & Orphan Removal in Relational Queries:

• **Why FetchType and OrphanRemoval Matter in Relations**

- When entities are related, two major concerns arise:
 - i. **When** should related **data** be **loaded**? → FetchType
 - ii. **When** should **unused** related **data** be **deleted**? → orphanRemoval
- They control performance and data consistency.

• FetchType (When Data is Loaded):

• **What is FetchType ?**

- FetchType defines how and when related entities are fetched from the database.
- Declared on relationships:
 - a) @ManyToOne(fetch = FetchType.LAZY)
 - b) @OneToMany(fetch = FetchType.LAZY)

• FetchType Types:

• **FetchType.EAGER :**

- Data is loaded **immediately** with parent entity.
@ManyToOne(fetch = FetchType.EAGER) // default
private Department department;
- Behavior
Employee e = repo.findById(1L);
- Hibernate executes:
SELECT * FROM employee
JOIN department ...
(or multiple queries)
- ✓ Always available
- ✗ Performance overhead
- ✗ N+1 risk

• **FetchType.LAZY :**

- Data is loaded **only when accessed**.
@ManyToOne(fetch = FetchType.LAZY) // default
private Department department;
- Behavior
Employee e = repo.findById(1L);
e.getDepartment(); // query fires here
- ✓ Better performance
- ✓ Smaller initial query
- ✗ Needs active transaction

3.9 FetchType and Orphan Removal in Relational Queries, N+1 Query Optimization

• Default Fetch Types:

Relationship	Default
@ManyToOne	EAGER
@OneToOne	EAGER
@OneToMany	LAZY
@ManyToMany	LAZY

- Always override defaults in production.

• FetchType in Relational Queries:

• **Fetch Join (Override Lazy/Eager) :**

```
@Query("SELECT e FROM Employee e JOIN FETCH e.department")  
List<Employee> findAllWithDepartment();
```

- ✓ One query
- ✓ No N+1
- ✓ Ignores FetchType temporarily

• Common Fetch Problems:

• **N+1 Query Problem :**

- N+1 happens when Hibernate runs **1 query** for **parent** + **N queries** for **children** inside a loop.
- N+1 problem occurs when Hibernate executes **one** query to load **parent entities** and additional queries for each **related entity** due to lazy loading.
- Fix:
 - ✓ **Fetch Join** : Load **parent** and **child** together in **ONE query** using **SQL JOIN**.
 - ✓ **DTO** : Instead of loading entities, you directly load only needed fields. Here no lazy relations, no persistence context dependency.

• **LazyInitializationException :**

- Accessing lazy fields outside transaction.
- Hibernate wanted to fetch related data, but the database connection (session) was already closed.
- Fix:
 - ✓ **@Transactional** : it keeps the session **open** for the method.
 - ✓ **Fetch join** : Fetch join initializes the lazy relationship before the session closes.
 - ✓ **DTO mapping** : Entities are converted into plain objects while the persistence context is still active, forcing lazy associations to be initialized.

3.9 FetchType and Orphan Removal in Relational Queries, N+1 Query Optimization

- What Is Orphan Removal in JPA?:

- **Orphan removal means:**

- If a child entity is removed from its parent's collection, it is **automatically deleted from the database**.

- **Example:**

```
@Entity
class Order {
    @OneToMany(
        mappedBy = "order",
        cascade = CascadeType.ALL,
        orphanRemoval = true
    )
    private List<Item> items;
}
```

- **Here:**

- Order → Parent
- Item → Child
- orphanRemoval = true → Auto-delete unused children

- **An orphan means,** A child entity that has no parent reference.

- **Example:** `Item.order = null`

- No parent → orphan → removed (if enabled).

- Orphan Removal vs Cascade:

- **Cascade**

- `cascade = CascadeType.REMOVE`
- Deletes children when **parent is deleted**.

- **Orphan Removal**

- `orphanRemoval = true`
- Deletes children when **parent removes reference**.